



**CSMSS**  
**CHH. SHAHU COLLEGE OF ENGINEERING,**  
**KANCHANWADI, CHHATRAPATI SAMBHAJINAGAR - 431002**

|                                                                 |                          |
|-----------------------------------------------------------------|--------------------------|
| <b>DEPARTMENT OF ARTIFICIAL INTELLIGENCE &amp; DATA SCIENCE</b> | <b>LABORATORY MANUAL</b> |
| <b>COURSE: INTRODUCTION TO OPERATING SYSTEM</b>                 | <b>INDEX</b>             |

| <b>Pr. No</b> | <b>Title</b>                                                                                                                                   | <b>Hardware / Software (Tools)</b>                                                                       | <b>Page No</b> |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|----------------|
| 1             | Study and execute basic UNIX/Linux commands for file management, directory handling, process monitoring, and permission control.               | <b>Hardware:</b> PC / Laptop<br><b>Software:</b> Linux Operating System (Ubuntu / Fedora)                |                |
| 2             | Write and execute shell programs for file handling operations and text processing using UNIX commands such as grep, awk, and sed.              | <b>Hardware:</b> PC / Laptop<br><b>Software:</b> Linux Operating System (Ubuntu / Fedora)                |                |
| 3             | Implement and analyze CPU scheduling algorithms such as First Come First Serve (FCFS), Shortest Job First (SJF), and Priority Scheduling.      | <b>Hardware:</b> PC / Laptop<br><b>Software:</b> Linux Operating System (Ubuntu / Fedora) , GCC Compiler |                |
| 4             | Study process creation and concurrent execution using system calls like fork() and vfork().                                                    | <b>Hardware:</b> PC / Laptop<br><b>Software:</b> Linux Operating System (Ubuntu / Fedora) , GCC Compiler |                |
| 5             | Implement multithreading using POSIX threads (pthreads) and analyse performance improvement compared to process-based execution.               | <b>Hardware:</b> PC / Laptop<br><b>Software:</b> Linux Operating System (Ubuntu / Fedora) , GCC Compiler |                |
| 6             | Implement and compare page replacement algorithms such as FIFO, LRU, and Optimal for effective memory management.                              | <b>Hardware:</b> PC / Laptop<br><b>Software:</b> Linux Operating System (Ubuntu / Fedora) , GCC Compiler |                |
| 7             | Implement memory allocation strategies (First Fit, Best Fit, and Worst Fit) and disk scheduling algorithms for efficient resource utilization. | <b>Hardware:</b> PC / Laptop<br><b>Software:</b> Linux Operating System (Ubuntu / Fedora) , GCC Compiler |                |
| 8             | Implement synchronization mechanisms using semaphores, mutex locks, and condition variables to avoid race conditions.                          | <b>Hardware:</b> PC / Laptop<br><b>Software:</b> Linux Operating System (Ubuntu / Fedora) , GCC Compiler |                |
| 9             | Implement classical operating system problems such as the Producer–Consumer problem and Banker’s algorithm for deadlock avoidance.             | <b>Hardware:</b> PC / Laptop<br><b>Software:</b> Linux Operating System (Ubuntu / Fedora) , GCC Compiler |                |
| 10            | Study advanced operating system concepts including parallel algorithms, kernel configuration, system utilities, and performance tuning.        | <b>Hardware:</b> PC / Laptop<br><b>Software:</b> Linux Operating System (Ubuntu / Fedora) ,              |                |

|                                                                                                                                                                                                                                              |                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
|  <p style="text-align: center;"><b>CSMSS</b><br/> <b>CHH. SHAHU COLLEGE OF ENGINEERING,</b><br/> <b>KANCHANWADI, CHHATRAPATI SAMBHAJINAGAR - 431002</b></p> |                          |
| <b>DEPARTMENT OF ARTIFICIAL INTELLIGENCE &amp; DATA SCIENCE</b>                                                                                                                                                                              | <b>LABORATORY MANUAL</b> |
| <b>COURSE: INTRODUCTION TO OPERATING SYSTEM</b>                                                                                                                                                                                              | <b>LAB NO. 01</b>        |

**Aim:** To study and execute basic UNIX/Linux commands for file management, directory handling, process monitoring, and permission control.

**Hardware Requirement:** PC/Laptop with minimum 4 GB RAM

**Software Requirement:** Linux Operating System (Ubuntu / Fedora)

**Tools / Libraries:** Terminal, Bash Shell

### Theory:

UNIX/Linux is a multi-user, multitasking operating system widely used in academic, research, and industrial environments. It provides a powerful Command Line Interface (CLI) through which users interact with the operating system using shell commands. These commands communicate with the kernel to perform operations such as file handling, process control, and permission management.

Unlike graphical interfaces, the command-line interface offers greater control, flexibility, and automation capabilities. Learning basic UNIX commands is essential for understanding how the operating system manages resources internally and forms the foundation for advanced topics such as shell scripting, system programming, and process management.

UNIX commands can be broadly classified as:

- **File Management Commands:**  
ls, cp, mv, rm – used to list, copy, move, and delete files.
- **Directory Management Commands:**  
mkdir, rmdir, cd – used to create, remove, and navigate directories.
- **Process Management Commands:**  
ps, top – used to monitor running processes and system performance.
- **Permission Management Commands:**  
chmod, chown – used to control access permissions and file ownership.

Understanding these commands helps users efficiently interact with the operating system and manage system resources effectively.

### Algorithm / Flow Chart:

1. Open the terminal window.
2. Execute basic file and directory-related commands.
3. Execute process monitoring commands.
4. Observe and analyze the system output.

### Reference Program:

```
$ pwd
$ mkdir oslab
$ cd oslab
$ touch file1.txt
$ ls -l
```

### Output:

- The current working directory is displayed.
- A new directory and file are created successfully.
- File details such as permissions and ownership are listed.

### Survey Question:

1. Why is Linux preferred for Operating System laboratories?
2. What is the role of the shell in UNIX/Linux?
3. Differentiate between absolute path and relative path.
4. What is a process ID (PID) and why is it important?
5. How does Linux support multi-user environment?

**Conclusion:** Basic UNIX/Linux commands provide a fundamental understanding of file systems, directory structures, and process management. Mastering these commands is essential for effective interaction with the operating system and forms a strong base for learning advanced operating system concepts.

**Prof. K. A. Kadam**  
Course Coordinator

**Dr. S. R. Zanwar**  
Head Of Department



**CSMSS**  
**CHH. SHAHU COLLEGE OF ENGINEERING,**  
**KANCHANWADI, CHHATRAPATI SAMBHAJINAGAR - 431002**

**DEPARTMENT OF ARTIFICIAL  
INTELLIGENCE & DATA SCIENCE**

**LABORATORY MANUAL**

**COURSE: INTRODUCTION TO OPERATING  
SYSTEM**

**LAB NO. 02**

**Aim:** To develop and execute shell programs for file handling and text processing using UNIX commands such as grep, awk, and sed.

**Hardware Requirement:** PC/Laptop with minimum 4 GB RAM

**Software Requirement:** Linux Operating System (Ubuntu / Fedora)

**Tools / Libraries:** Terminal, Bash Shell

### **Theory:**

Shell scripting is a powerful feature of UNIX/Linux operating systems that allows users to automate repetitive tasks by combining multiple commands into a single script. A shell script is a text file containing a sequence of commands executed by the shell.

File handling in shell scripting involves operations such as reading file contents, searching specific data, filtering records, and modifying text. UNIX provides specialized text-processing utilities that make these tasks efficient and fast.

- **grep** (Global Regular Expression Print): Used to search for specific patterns or keywords within a file.
- **awk**: A pattern scanning and processing language mainly used for handling structured data and generating formatted output.
- **sed** (Stream Editor): Used to perform text transformations such as substitution, deletion, and insertion in a stream of data.

Using these commands within shell scripts enables efficient data processing, log analysis, and automation of system tasks. Understanding shell scripting and text processing commands forms the foundation for advanced topics such as system programming, process automation, and server management.

### **Algorithm / Flow Chart:**

1. Create a shell script file.
2. Accept filename as input from the user.
3. Search for a specific keyword using grep.
4. Display matching lines from the file.

### Reference Program:

```
#!/bin/bash
# Create a sample file
echo "Operating System Lab" > os.txt
echo "Shell scripting in Linux" >> os.txt
echo "Text processing using grep awk sed" >> os.txt

# Display file content
cat os.txt

# Use grep to search a word
grep "Linux" os.txt

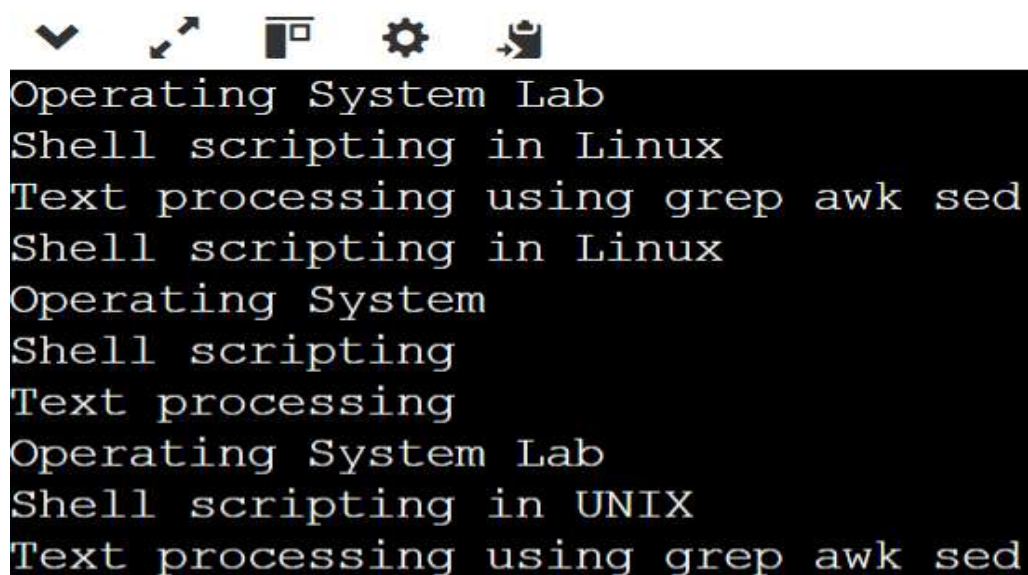
# Use awk to display first two words of each line
awk '{print $1,$2}' os.txt

# Use sed to replace a word
sed 's/Linux/UNIX/' os.txt
```

### Commands to execute the program:

```
chmod +x lab02.sh
./lab02.sh
```

### Output:



```
Operating System Lab
Shell scripting in Linux
Text processing using grep awk sed
Shell scripting in Linux
Operating System
Shell scripting
Text processing
Operating System Lab
Shell scripting in UNIX
Text processing using grep awk sed
```

### Survey Question:

1. What is a shell script?
2. Why is shell scripting important in UNIX/Linux?
3. Differentiate between awk and sed.

4. What is the use of grep command?
5. How does shell scripting help system administrators?

**Conclusion:** Shell scripting simplifies file handling and text processing tasks by automating command execution. The use of tools like grep, awk, and sed improves efficiency, reduces manual effort, and enhances productivity in UNIX/Linux environments.

**Prof. K. A. Kadam**  
Course Coordinator

**Dr. S. R. Zanwar**  
Head Of Department



**CSMSS**  
**CHH. SHAHU COLLEGE OF ENGINEERING,**  
KANCHANWADI, CHHATRAPATI SAMBHAJINAGAR - 431002

|                                                                     |                          |
|---------------------------------------------------------------------|--------------------------|
| <b>DEPARTMENT OF ARTIFICIAL<br/>INTELLIGENCE &amp; DATA SCIENCE</b> | <b>LABORATORY MANUAL</b> |
| <b>COURSE: INTRODUCTION TO OPERATING<br/>SYSTEM</b>                 | <b>LAB NO. 03</b>        |

**Aim:** To implement and analyse CPU scheduling algorithms such as First Come First Serve (FCFS), Shortest Job First (SJF), and Priority Scheduling.

**Hardware Requirement:** PC/Laptop with a minimum 4 GB RAM

**Software Requirement:** Linux Operating System (Ubuntu / Fedora)

**Tools / Libraries:** Terminal, Bash Shell

### Theory:

CPU scheduling is a fundamental concept of a **multiprogramming operating system**. In a multiprogramming environment, multiple processes reside in the ready queue, and the operating system must decide which process should be allocated the CPU next. This decision is made by the **CPU scheduler**.

The choice of scheduling algorithm has a direct impact on system performance parameters such as:

- **Waiting Time** – time a process waits in the ready queue
- **Turnaround Time** – total time taken from submission to completion
- **Throughput** – number of processes completed per unit time

Common CPU scheduling algorithms include:

- **First Come First Serve (FCFS):**  
Processes are executed in the order of their arrival. It is simple to implement but may lead to long waiting times.
- **Shortest Job First (SJF):**  
The process with the shortest CPU burst time is selected next. It minimizes average waiting time but may cause starvation.
- **Priority Scheduling:**  
CPU is allocated based on priority values assigned to processes. Higher priority processes execute first.

Understanding these algorithms helps in analyzing how operating systems optimize CPU utilization and improve overall system efficiency.

**Algorithm / Flow Chart:**

1. Read the number of processes.
2. Read burst time for each process.
3. Set waiting time of first process to zero.
4. Calculate waiting time of remaining processes.
5. Calculate turnaround time for each process.
6. Display waiting time and turnaround time.

### Reference Program:

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[10], wt[10], tat[10];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Enter burst time of process %d: ", i+1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;
    for(i = 1; i < n; i++) {
        wt[i] = wt[i-1] + bt[i-1];
    }

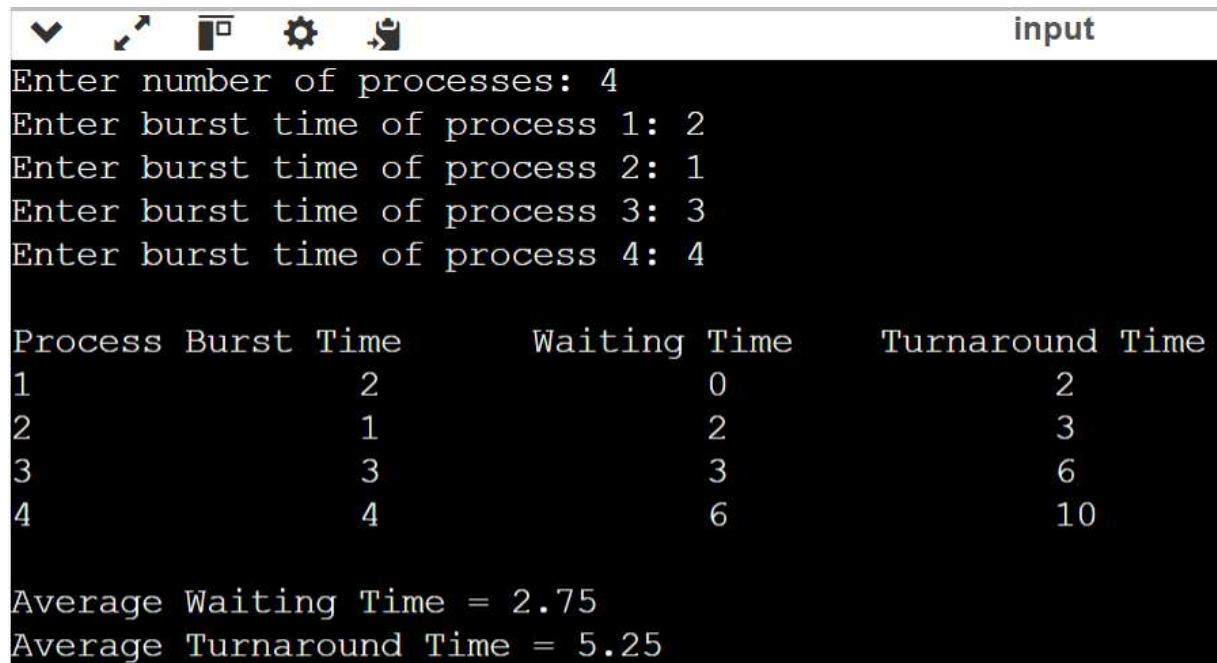
    for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avgwt += wt[i];
        avgtat += tat[i];
    }

    avgwt /= n;
    avgtat /= n;

    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for(i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avgwt);
    printf("\nAverage Turnaround Time = %.2f\n", avgtat);

    return 0;
}
```

**Output:**

```
input
Enter number of processes: 4
Enter burst time of process 1: 2
Enter burst time of process 2: 1
Enter burst time of process 3: 3
Enter burst time of process 4: 4

Process Burst Time      Waiting Time      Turnaround Time
1          2              0                 2
2          1              2                 3
3          3              3                 6
4          4              6                10

Average Waiting Time = 2.75
Average Turnaround Time = 5.25
```

**Survey Question:**

1. What is CPU scheduling?
2. Differentiate between preemptive and non-preemptive scheduling.
3. What is starvation in CPU scheduling?
4. Which scheduling algorithm minimizes average waiting time?
5. Why FCFS is considered simple but inefficient?

**Conclusion:**

CPU scheduling algorithms play a crucial role in determining the performance of an operating system. The selection of an appropriate scheduling algorithm directly affects CPU utilization, waiting time, and overall system efficiency. Different algorithms are suitable for different system requirements.

**Prof. K. A. Kadam**  
Course Coordinator

**Dr. S. R. Zanwar**  
Head Of Department



**CSMSS**  
**CHH. SHAHU COLLEGE OF ENGINEERING,**  
KANCHANWADI, CHHATRAPATI SAMBHAJINAGAR - 431002

|                                                                     |                          |
|---------------------------------------------------------------------|--------------------------|
| <b>DEPARTMENT OF ARTIFICIAL<br/>INTELLIGENCE &amp; DATA SCIENCE</b> | <b>LABORATORY MANUAL</b> |
| <b>COURSE: INTRODUCTION TO OPERATING<br/>SYSTEM</b>                 | <b>LAB NO. 04</b>        |

**Aim:** To study and implement process creation in UNIX/Linux using the fork() system call and to understand parent-child process behavior.

**Hardware Requirement:** PC/Laptop with minimum 4 GB RAM

**Software Requirement:** Linux Operating System (Ubuntu / Fedora)

**Tools / Libraries:** GCC Compiler, Terminal, Bash Shell

### Theory:

In UNIX/Linux operating systems, a process is an instance of a program in execution. Process creation is a fundamental operating system function that allows multiple programs to run concurrently. The fork() system call is used to create a new process.

When a process calls fork(), a child process is created which is an exact copy of the parent process, except for the process ID. Both processes execute concurrently after the fork() call.

The return value of fork() helps distinguish between the parent and child processes:

- fork() returns 0 in the child process
- fork() returns child's PID in the parent process
- fork() returns -1 if process creation fails

Understanding process creation helps in learning multitasking, process scheduling, and inter-process communication in operating systems.

### Algorithm / Flow Chart:

1. Start the program.
2. Call the fork() system call.
3. Check the return value of fork().
4. If return value is 0, execute child process code.
5. If return value is greater than 0, execute parent process code.
6. Display process IDs.
7. End the program.

### Reference Program:

```
#include <stdio.h>
#include <unistd.h>

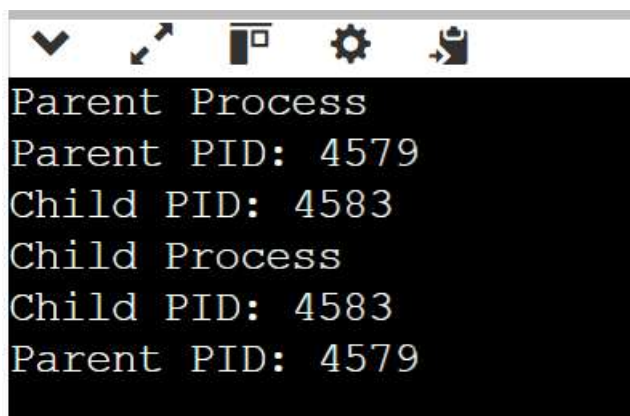
int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Process creation failed.\n");
    }
    else if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
    }
    else {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }
    return 0;
}
```

### Output:

- A child process is created successfully.
- Parent and child processes execute independently.
- Different process IDs are displayed for parent and child.
- Process creation using fork() is demonstrated.



```
Parent Process
Parent PID: 4579
Child PID: 4583
Child Process
Child PID: 4583
Parent PID: 4579
```

### Survey Question:

1. What is a process?
2. Explain the working of fork() system call.
3. How does the parent process identify the child process?
4. What is the difference between fork() and exec()?
5. Why is process creation important in an operating system?

**Conclusion:**

The fork() system call enables the creation of new processes in UNIX/Linux systems. Through this experiment, the concept of parent and child processes was clearly understood. Process creation is essential for multitasking and efficient resource utilization in operating systems.

**Prof. K. A. Kadam**  
**Course Coordinator**

**Dr. S. R. Zanwar**  
**Head Of Department**



**CSMSS**  
**CHH. SHAHU COLLEGE OF ENGINEERING,**  
**KANCHANWADI, CHHATRAPATI SAMBHAJINAGAR - 431002**

**DEPARTMENT OF ARTIFICIAL  
INTELLIGENCE & DATA SCIENCE**

**LABORATORY MANUAL**

**COURSE: INTRODUCTION TO OPERATING  
SYSTEM**

**LAB NO. 05**

**Aim:** To study and implement multithreading in UNIX/Linux using POSIX Threads (pthread) and understand concurrent execution of threads.

**Hardware Requirement:** PC/Laptop with minimum 4 GB RAM

**Software Requirement:** Linux Operating System (Ubuntu / Fedora)

**Tools / Libraries:** GCC Compiler, Terminal, POSIX Threads Library (pthread)

### **Theory:**

A thread is the smallest unit of execution within a process. Multithreading allows multiple threads to exist within a single process, sharing the same memory space and resources. This improves application performance, responsiveness, and efficient CPU utilization.

In UNIX/Linux systems, multithreading is supported through POSIX Threads (pthreads). Each thread has its own execution path but shares code, data, and files with other threads of the same process.

Advantages of multithreading include:

- Improved application performance
- Efficient use of CPU resources
- Reduced context switching overhead
- Better responsiveness in interactive applications

Understanding multithreading is essential for developing modern applications such as web servers, parallel programs, and real-time systems.

### **Algorithm / Flow Chart:**

1. Start the program.
2. Create threads using `pthread_create()`.
3. Execute thread functions concurrently.
4. Wait for thread completion using `pthread_join()`.
5. Display output from each thread.
6. Terminate the program.

**Reference Program:**

```
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    int id = *((int*)arg);
    printf("Thread %d is executing\n", id);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    pthread_create(&t1, NULL, thread_function, &id1);
    pthread_create(&t2, NULL, thread_function, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("All threads have finished execution\n");

    return 0;
}
```

**Procedure:**

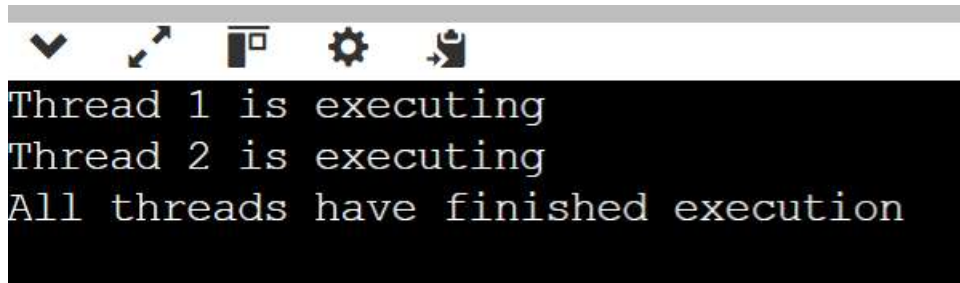
1. Open the terminal.
2. Create a C program file using a text editor.
3. Compile the program with pthread support.
4. Execute the program.
5. Observe concurrent execution of threads.

**Compilation Command:**

```
gcc lab5.c -o lab5 -lpthread
```

**Output:**

- Multiple threads are created successfully.
- Threads execute concurrently.
- Output from each thread is displayed.
- Main thread waits until all threads complete execution.



```
Thread 1 is executing
Thread 2 is executing
All threads have finished execution
```

**Survey Question:**

1. What is a thread?
2. Differentiate between process and thread.
3. What is the role of `pthread_create()`?
4. Why is `pthread_join()` important?
5. List advantages of multithreading.

**Conclusion:**

Multithreading enables concurrent execution of multiple threads within a process, leading to better performance and resource utilization. This experiment demonstrated thread creation and synchronization using POSIX Threads, which is a key concept in modern operating systems.

**Prof. K. A. Kadam**  
Course Coordinator

**Dr. S. R. Zanwar**  
Head Of Department



|                                                                     |                          |
|---------------------------------------------------------------------|--------------------------|
| <b>DEPARTMENT OF ARTIFICIAL<br/>INTELLIGENCE &amp; DATA SCIENCE</b> | <b>LABORATORY MANUAL</b> |
| <b>COURSE: INTRODUCTION TO OPERATING<br/>SYSTEM</b>                 | <b>LAB NO. 06</b>        |

**Aim:** To study and implement Page Replacement Algorithms such as FIFO, Least Recently Used (LRU), and Optimal, and to compare their performance in terms of page faults.

**Hardware Requirement:** PC/Laptop with minimum 4 GB RAM

**Software Requirement:** Linux Operating System (Ubuntu / Fedora)

**Tools / Libraries:** GCC Compiler, Terminal, Text Editor

### Theory:

In a virtual memory system, a **page fault** occurs when a program accesses a page that is not currently present in main memory. Page replacement algorithms determine which page should be removed from memory when a new page needs to be loaded and all memory frames are full.

Common page replacement algorithms include:

- **First In First Out (FIFO):** The oldest page in memory is replaced first. It is simple but may suffer from Belady's anomaly.
- **Least Recently Used (LRU):** The page that has not been used for the longest period of time is replaced. It performs better than FIFO but requires tracking of page usage.
- **Optimal Page Replacement:** Replaces the page that will not be used for the longest time in the future. It provides the minimum number of page faults but is not practically implementable.

Studying these algorithms helps in understanding memory management strategies used by operating systems.

### Algorithm / Flow Chart:

- Step 1:** Read number of frames
- Step 2:** Read reference string
- Step 3:** Initialize memory frames as empty
- Step 4:** Apply selected page replacement algorithm
- Step 5:** Count page faults
- Step 6:** Display total page faults

### Reference Program:

```
#include <stdio.h>

int main() {
    int frames, pages, i, j, k = 0, faults = 0;
    int ref[20], mem[10];

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    printf("Enter number of pages: ");
    scanf("%d", &pages);

    printf("Enter reference string:\n");
    for (i = 0; i < pages; i++)
        scanf("%d", &ref[i]);

    for (i = 0; i < frames; i++)
        mem[i] = -1;

    for (i = 0; i < pages; i++) {
        int found = 0;
        for (j = 0; j < frames; j++) {
            if (mem[j] == ref[i]) {
                found = 1;
                break;
            }
        }

        if (!found) {
            mem[k] = ref[i];
            k = (k + 1) % frames;
            faults++;
        }
    }

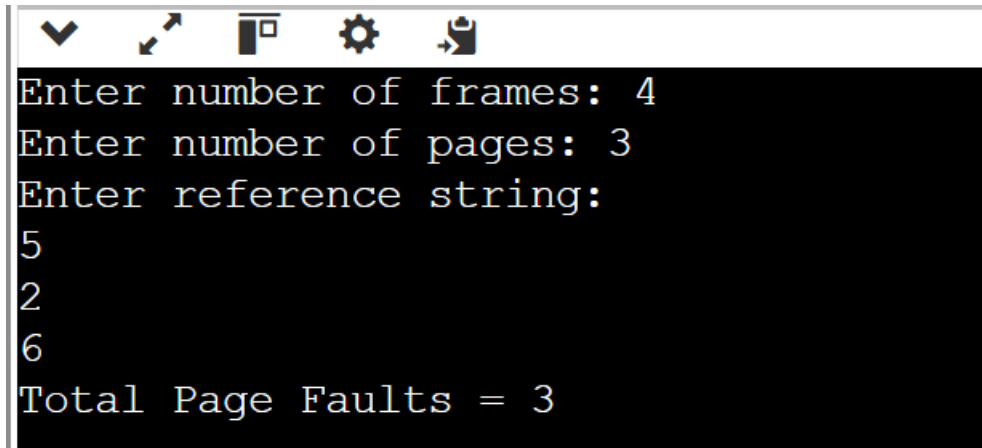
    printf("Total Page Faults = %d\n", faults);
    return 0;
}
```

### Procedure:

1. Open the terminal.
2. Create a C program file using a text editor.
3. Compile the program using GCC compiler.
4. Execute the program.
5. Enter the number of frames and page reference string.
6. Observe the number of page faults.

**Output:**

- Number of frames and page reference string are accepted.
- FIFO page replacement algorithm is applied.
- Page faults are calculated correctly.
- Total number of page faults is displayed.



```
Enter number of frames: 4
Enter number of pages: 3
Enter reference string:
5
2
6
Total Page Faults = 3
```

**Survey Question:**

1. What is a page fault?
2. Explain FIFO page replacement algorithm.
3. What is Belady's anomaly?
4. Why is LRU better than FIFO?
5. Why is Optimal page replacement not practically implementable?

**Conclusion:**

Page replacement algorithms play a crucial role in memory management. This experiment demonstrated the FIFO page replacement technique and highlighted the importance of choosing efficient algorithms to minimize page faults and improve system performance.

**Prof. K. A. Kadam**  
Course Coordinator

**Dr. S. R. Zanwar**  
Head Of Department



|                                                                     |                          |
|---------------------------------------------------------------------|--------------------------|
| <b>DEPARTMENT OF ARTIFICIAL<br/>INTELLIGENCE &amp; DATA SCIENCE</b> | <b>LABORATORY MANUAL</b> |
| <b>COURSE: INTRODUCTION TO OPERATING<br/>SYSTEM</b>                 | <b>LAB NO. 07</b>        |

**Aim:** To study and implement memory allocation techniques such as First Fit, Best Fit, and Worst Fit, and to understand disk scheduling algorithms used in operating systems.

**Hardware Requirement:** PC/Laptop with minimum 4 GB RAM

**Software Requirement:** Linux Operating System (Ubuntu / Fedora)

**Tools / Libraries:** GCC Compiler, Terminal, Text Editor

### Theory:

Memory management is a critical function of an operating system that controls and coordinates computer memory, assigning portions to programs as needed. Efficient memory allocation improves system performance and reduces fragmentation.

Common memory allocation techniques include:

- **First Fit:** Allocates the first available memory block that is large enough to satisfy the request.
- **Best Fit:** Allocates the smallest memory block that is sufficient for the process, reducing internal fragmentation.
- **Worst Fit:** Allocates the largest available memory block, leaving large remaining blocks for future requests.

Disk scheduling algorithms determine the order in which disk I/O requests are serviced. Proper disk scheduling minimizes seek time and improves disk performance. Common disk scheduling algorithms include **FCFS**, **SSTF**, **SCAN**, and **C-SCAN**.

### Algorithm / Flow Chart:

#### Memory Allocation:

1. Read memory block sizes.
2. Read process sizes.
3. Apply First Fit / Best Fit / Worst Fit algorithm.
4. Allocate memory and display results.

#### Disk Scheduling:

1. Read disk request queue.
2. Read initial head position.
3. Apply selected disk scheduling algorithm.
4. Calculate total head movement.

### Reference Program:

```
#include <stdio.h>

int main() {
    int i, j, nb, np;
    int b[10], p[10], allocation[10];

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);

    printf("Enter number of processes: ");
    scanf("%d", &np);

    printf("Enter size of memory blocks:\n");
    for (i = 0; i < nb; i++)
        scanf("%d", &b[i]);

    printf("Enter size of processes:\n");
    for (i = 0; i < np; i++) {
        scanf("%d", &p[i]);
        allocation[i] = -1;
    }

    for (i = 0; i < np; i++) {
        for (j = 0; j < nb; j++) {
            if (b[j] >= p[i]) {
                allocation[i] = j;
                b[j] -= p[i];
                break;
            }
        }
    }

    printf("\nProcess No\tProcess Size\tBlock No\n");
    for (i = 0; i < np; i++) {
        printf("%d\t\t%d\t\t", i + 1, p[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }

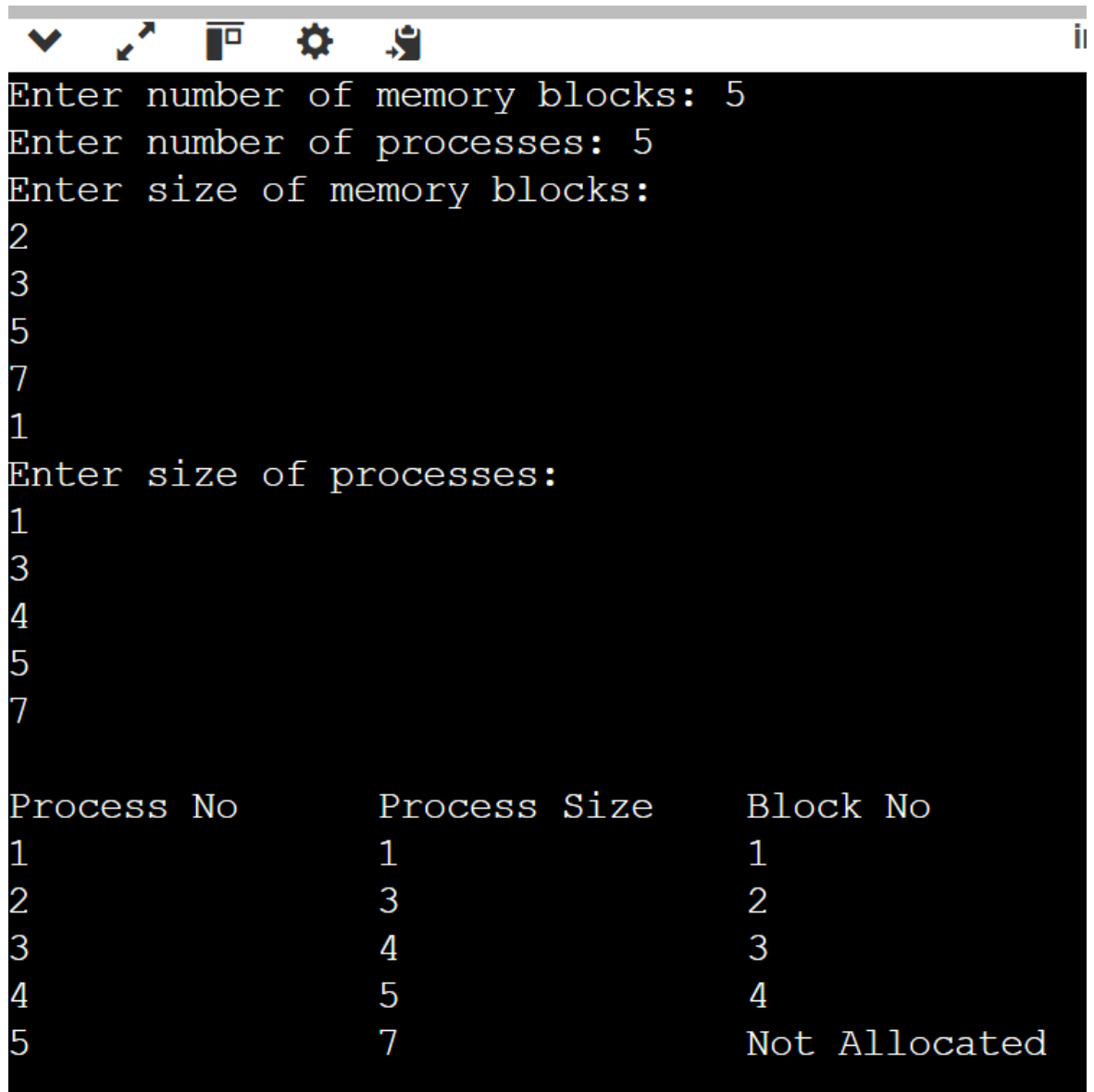
    return 0;
}
```

### Procedure:

1. Open the terminal.
2. Create a C program file using a text editor.
3. Compile the program using GCC compiler.
4. Execute the program.
5. Enter memory block sizes and process sizes.
6. Observe memory allocation results.

### Output:

- Memory block sizes and process sizes are accepted.
- First Fit memory allocation algorithm is applied.
- Processes are allocated to suitable memory blocks.
- Unallocated processes are clearly indicated.



```
Enter number of memory blocks: 5
Enter number of processes: 5
Enter size of memory blocks:
2
3
5
7
1
Enter size of processes:
1
3
4
5
7

Process No      Process Size      Block No
1                1                 1
2                3                 2
3                4                 3
4                5                 4
5                7                 Not Allocated
```

### Survey Question:

1. What is memory allocation?
2. Differentiate between First Fit, Best Fit, and Worst Fit.
3. What is fragmentation?
4. What is disk scheduling and why is it required?
5. Which disk scheduling algorithm minimizes seek time?

**Conclusion:**

Memory allocation and disk scheduling are essential operating system functions that directly affect system performance. This experiment demonstrated First Fit memory allocation and introduced disk scheduling concepts, highlighting the importance of efficient resource management.

**Prof. K. A. Kadam**  
**Course Coordinator**

**Dr. S. R. Zanwar**  
**Head Of Department**



|                                                                     |                          |
|---------------------------------------------------------------------|--------------------------|
| <b>DEPARTMENT OF ARTIFICIAL<br/>INTELLIGENCE &amp; DATA SCIENCE</b> | <b>LABORATORY MANUAL</b> |
| <b>COURSE: INTRODUCTION TO OPERATING<br/>SYSTEM</b>                 | <b>LAB NO. 08</b>        |

**Aim:** To study and implement process synchronization using Semaphore and Mutex mechanisms to avoid race conditions in concurrent execution.

**Hardware Requirement:** PC/Laptop with minimum 4 GB RAM

**Software Requirement:** Linux Operating System (Ubuntu / Fedora)

**Tools / Libraries:** GCC Compiler, Terminal, POSIX Threads Library (pthread), Semaphore Library

### Theory:

In a multitasking environment, multiple processes or threads may access shared resources simultaneously. This can lead to race conditions, where the final outcome depends on the order of execution. Process synchronization ensures that shared resources are accessed in a controlled and orderly manner.

Two commonly used synchronization mechanisms are:

- **Semaphore:** A semaphore is a signalling mechanism used to control access to shared resources. It can be binary or counting and is implemented using wait() and signal() operations.
- **Mutex (Mutual Exclusion):** A mutex allows only one thread at a time to enter the critical section. It provides ownership-based locking, making it suitable for thread synchronization.

Proper synchronization prevents data inconsistency, deadlock, and starvation, making it an essential concept in operating systems.

### Algorithm / Flow Chart:

1. Initialize semaphore or mutex.
2. Create multiple threads.
3. Allow only one thread to enter the critical section at a time.
4. Perform operations on shared resource.
5. Release semaphore or unlock mutex.
6. Terminate threads and program.

**Reference Program:**

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t mutex;
int count = 0;

void* increment(void* arg) {
    pthread_mutex_lock(&mutex);
    count++;
    printf("Count value: %d\n", count);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&mutex);

    return 0;
}
```

**Procedure:**

1. Open the terminal.
2. Create a C program file using a text editor.
3. Compile the program with pthread support.
4. Execute the program.
5. Observe synchronized access to shared variable.

**Compilation Command:** gcc lab8.c -o lab8 -lpthread

**Output:**

- Multiple threads are created successfully.
- Shared variable is accessed one thread at a time.
- Race condition is avoided using mutex.
- Correct and synchronized output is obtained.

**Survey Question:**

1. What is process synchronization?
2. What is a race condition?
3. Differentiate between semaphore and mutex.
4. Why is synchronization required in operating systems?
5. What happens if synchronization is not used?

**Conclusion:**

Process synchronization is essential to maintain data consistency in concurrent execution environments. This experiment demonstrated the use of mutex for synchronizing threads and preventing race conditions, highlighting its importance in operating systems.

**Prof. K. A. Kadam**  
Course Coordinator

**Dr. S. R. Zanwar**  
Head Of Department



|                                                                     |                          |
|---------------------------------------------------------------------|--------------------------|
| <b>DEPARTMENT OF ARTIFICIAL<br/>INTELLIGENCE &amp; DATA SCIENCE</b> | <b>LABORATORY MANUAL</b> |
| <b>COURSE: INTRODUCTION TO OPERATING<br/>SYSTEM</b>                 | <b>LAB NO. 09</b>        |

**Aim:** To study and implement the Producer–Consumer problem using synchronisation concepts and to understand Deadlock Avoidance using Banker’s Algorithm.

**Hardware Requirement:** PC/Laptop with minimum 4 GB RAM

**Software Requirement:** Linux Operating System (Ubuntu / Fedora)

**Tools / Libraries:** GCC Compiler, Terminal, POSIX Threads Library (pthread),

### Theory:

In an operating system, multiple processes often need to share limited resources. Improper handling of shared resources can lead to problems such as **race conditions** and **deadlocks**.

#### Producer–Consumer Problem:

The Producer–Consumer problem is a classic synchronization problem where:

- The **producer** generates data and places it into a shared buffer.
- The **consumer** removes data from the shared buffer.

Synchronization mechanisms such as semaphores or mutex are used to ensure that:

- The producer does not add data to a full buffer.
- The consumer does not remove data from an empty buffer.

#### Banker’s Algorithm:

Banker’s Algorithm is a **deadlock avoidance** technique that checks whether granting a resource request will leave the system in a **safe state**. It ensures that deadlock never occurs by carefully allocating resources based on maximum needs.

Understanding these concepts is essential for designing safe and efficient operating systems.

### Algorithm / Flow Chart:

#### Producer–Consumer:

1. Initialize shared buffer and synchronization variables.
2. Create producer and consumer threads.
3. Producer produces items and places them in buffer.
4. Consumer consumes items from buffer.
5. Synchronize access to avoid race conditions.

**Banker's Algorithm:**

1. Read number of processes and resources.
2. Read allocation, maximum, and available matrices.
3. Calculate need matrix.
4. Check for safe sequence.
5. Display whether system is in safe or unsafe state.

**Reference Program:**

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex;
int buffer = 0;

void* producer(void* arg) {
    pthread_mutex_lock(&mutex);
    buffer++;
    printf("Producer produced item. Buffer = %d\n", buffer);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

void* consumer(void* arg) {
    pthread_mutex_lock(&mutex);
    buffer--;
    printf("Consumer consumed item. Buffer = %d\n", buffer);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main() {
    pthread_t p, c;

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    pthread_mutex_destroy(&mutex);
    return 0;
}
```

**Procedure:**

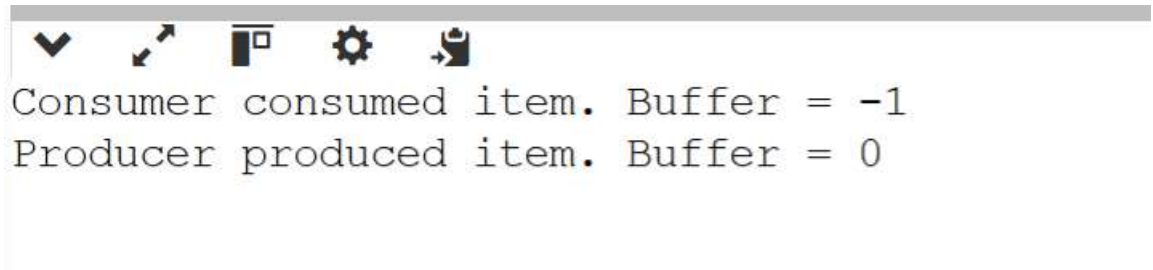
1. Open the terminal.
2. Create a C program file using a text editor.
3. Compile the program with pthread support.

4. Execute the program.
5. Observe producer and consumer execution.

**Compilation Command:** gcc lab9.c -o lab9 -lpthread

#### Output:

- Producer and consumer threads are created successfully.
- Shared buffer is accessed in a synchronized manner.
- Race condition is avoided.
- Producer–Consumer problem is demonstrated successfully.



```
Consumer consumed item. Buffer = -1
Producer produced item. Buffer = 0
```

#### Survey Question:

1. What is the Producer–Consumer problem?
2. Why is synchronization required in Producer–Consumer?
3. What is a deadlock?
4. Explain the concept of safe state in Banker's Algorithm.
5. How does Banker's Algorithm avoid deadlock?

#### Conclusion:

The Producer–Consumer problem illustrates the importance of synchronization in shared resource access, while Banker's Algorithm demonstrates how deadlock can be avoided through safe resource allocation. Both concepts are fundamental to operating system design and resource management.

**Prof. K. A. Kadam**  
Course Coordinator

**Dr. S. R. Zanwar**  
Head Of Department



|                                                                 |                          |
|-----------------------------------------------------------------|--------------------------|
| <b>DEPARTMENT OF ARTIFICIAL INTELLIGENCE &amp; DATA SCIENCE</b> | <b>LABORATORY MANUAL</b> |
| <b>COURSE: INTRODUCTION TO OPERATING SYSTEM</b>                 | <b>LAB NO. 10</b>        |

**Aim:** To study advanced operating system concepts such as parallel processing, kernel configuration, system monitoring, and performance tuning in UNIX/Linux environment.

**Hardware Requirement:** PC/Laptop with minimum 4 GB RAM

**Software Requirement:** Linux Operating System (Ubuntu / Fedora)

**Tools / Libraries:** Terminal, Bash Shell, System Monitoring Utilities

### **Theory:**

Modern operating systems are designed to support advanced features such as parallel processing, efficient kernel management, and system performance optimization. These features ensure maximum utilization of hardware resources and improved system responsiveness.

- **Parallel Processing:** Parallel processing allows multiple tasks or threads to execute simultaneously on multi-core processors, improving throughput and performance.
- **Kernel Configuration:** The kernel is the core component of the operating system responsible for managing hardware and system resources. Kernel configuration allows enabling or disabling features to optimize system behavior.
- **System Monitoring:** Tools such as top, htop, and vmstat are used to monitor CPU usage, memory utilization, and running processes.
- **Performance Tuning:** Performance tuning involves adjusting system parameters to improve efficiency, reduce latency, and enhance overall system performance.

Understanding these concepts is essential for system administrators and developers working with large-scale and high-performance systems.

### **Algorithm / Flow Chart:**

1. Open the terminal.
2. Execute system monitoring commands.
3. Observe CPU, memory, and process statistics.
4. Analyze system performance parameters.
5. Record observations.

**Reference Program:**

```
$ uname -a  
$ top  
$ ps -ef  
$ free -h  
$ df -h
```

**Procedure:**

1. Open the terminal window.
2. Execute each system monitoring command one by one.
3. Observe CPU usage, memory status, and running processes.
4. Analyze disk space and system information.

**Output:**

- System and kernel information is displayed.
- Active processes and CPU utilization are shown.
- Memory usage details are displayed.
- Disk space usage is displayed.
- Overall system performance is observed.

**Survey Question:**

1. What is the role of the kernel in an operating system?
2. What is parallel processing?
3. How does system monitoring help administrators?
4. What is performance tuning?
5. Why is Linux preferred for high-performance systems?

**Conclusion:**

Advanced operating system concepts such as parallel processing, kernel configuration, and performance monitoring play a crucial role in efficient system operation. This experiment provided practical exposure to system monitoring and performance analysis in a Linux environment.

**Prof. K. A. Kadam**  
Course Coordinator

**Dr. S. R. Zanwar**  
Head Of Department